Gestion de chantiers

Application desktop Java de gestion de chantiers

Phase 1 : Conception

Exigences, modélisation UML, architecture et maquettes

Équipe : Abdelghafour Agdz · Elhoucine Araay · Mouhssine Tazrakine
Fatima Zahra Bachkou · Abdessamad Bouderqa · Achraf Nait Bourdou

GitHub : https://github.com/Elhoucine-Araay/Group-Project-Java-Application

Synthèse du projet : Gestion de chantiers

Contexte et objectif

Nous développons une application de bureau en Java destinée à la gestion de chantiers. L'objectif de cette première phase était de concevoir l'interface et les fonctionnalités de l'application avant sa réalisation.

Travail réalisé

Cahier des charges : définit le périmètre et deux rôles distincts : le « chef de département » (qui supervise l'ensemble des projets et leurs responsables) et le « chef de projet » (qui assure la gestion opérationnelle sur le terrain).

Cas d'utilisation : 20 cas d'utilisation ont été identifiés, incluant un acteur abstrait commun pour les actions partagées par les deux rôles.

Diagramme de classes : le modèle intègre les entités issues du cahier des charges : utilisateurs, chantiers, tâches, employés, suivi des présences, matériaux, stocks et engins.

Diagrammes de séquence : cinq flux principaux ont été modélisés (connexion, suivi des présences, gestion des engins et des stocks).

Architecture : l'application repose sur trois couches : JavaFX pour l'interface graphique ; une couche de services pour la logique métier et la gestion des droits d'accès (définissant les rôles et privilèges des utilisateurs) ; et une couche DAO (Data Access Object) utilisant JDBC pour interagir avec une base de données PostgreSQL.

Maquettes : cinq visuels ont été créés, couvrant les écrans suivants : connexion, tableau de bord, liste des projets, détails d'un projet et suivi des présences.

Principaux choix de conception

- Définition de deux rôles, avec des droits d'accès contrôlés au niveau de la couche de services.
- Suivi des stocks avec traçabilité de chaque mouvement.
- Impossibilité d'affecter des engins en cours de maintenance ou en panne.
- Enregistrement unique de présence par employé, par chantier et par jour.
- Utilisation de technologies libres et open source (Java, JavaFX, PostgreSQL), choisies notamment pour limiter les coûts.

Travail restant à effectuer

- Finaliser les maquettes et concevoir les écrans manquants (tâches, stocks, engins, employés, documents, rapports, utilisateurs).

Gestion de chantiers

Cahier des charges

Application desktop de gestion de chantiers

Équipe :

- Abdelghafour Agdz
- Elhoucine Araay
- Mouhssine Tazrakine
- Fatima Zahra Bachkou
- Abdessamad Bouderqa
- Achraf Nait Bourdou

---

1. Introduction

Ce document décrit les exigences fonctionnelles et non fonctionnelles de l'application desktop **Construction Site Management**, dédiée à la gestion de chantiers de construction. Il précise le périmètre du projet, les acteurs, les fonctionnalités attendues, les contraintes techniques et les critères de qualité, de sécurité et de fiabilité à respecter.

Il sert de référence unique pour la conception UML, les maquettes, l'architecture, la base de données PostgreSQL et les tests. Chaque exigence porte un identifiant (EF, ENF, RG) repris dans les autres livrables.

2. Périmètre du projet

2.1. Inclus dans le projet

- Authentification et gestion de deux rôles : chef de département et chef de projet.
- Gestion des chantiers (création, modification, clôture), de leur budget et de leur avancement.
- Gestion des tâches, des employés, de leurs affectations et de leurs présences.
- Gestion des matériaux, du stock par chantier, des mouvements de stock et des alertes de seuil.
- Gestion des machines, de leur état, de leurs affectations et de leur maintenance.
- Gestion des documents liés à un chantier.
- Tableau de bord, génération, consultation et export de rapports d'avancement.

2.2. Hors périmètre (version actuelle)

- Application web ou mobile : le projet est une application desktop.
- Gestion de la paie, de la comptabilité détaillée et des factures fournisseurs.
- Pointage biométrique ou géolocalisé.
- Récupération de mot de passe par e-mail (le lien « Forgot password? » de la maquette de connexion est donc à retirer ou à documenter comme évolution future).
- Gestion multilingue : l'interface est rédigée dans une seule langue (à choisir et appliquer partout, voir ENF05).

3. Acteurs et rôles

Acteur	Description	Droits principaux
Chef de département	Responsable de plusieurs chantiers. Supervise l'activité globale.	Gérer les utilisateurs et les chantiers (dont budget total et chef de projet), consulter l'avancement, le tableau de bord et les rapports, exporter les rapports.
Chef de projet	Responsable opérationnel d'un ou plusieurs chantiers sur le terrain.	Gérer les tâches, employés, affectations, présences, matériaux, stock, machines, maintenance et documents ; générer, consulter et exporter les rapports ; consulter le tableau de bord.
Employé (ouvrier)	Personnel de chantier. **N'est pas un utilisateur** du système : il est enregistré et pointé par le chef de projet.	Aucun accès à l'application.

Les deux rôles utilisateurs héritent d'une même entité **Utilisateur** (e-mail, mot de passe haché, rôle). Les droits sont contrôlés par la couche Services (RG08).

4. Exigences fonctionnelles

Priorité : Haute = indispensable à la version livrée ; Moyenne = attendue mais non bloquante. CD = chef de département, CP = chef de projet. Les cas d'utilisation (UC) renvoient au diagramme de cas d'utilisation du dossier de conception.

Exigence	Description
Authentification	Se connecter avec e-mail et mot de passe, se déconnecter. Message d'erreur en cas d'échec.
Gestion des utilisateurs	Créer, modifier, désactiver des comptes et attribuer un rôle.
Gestion des chantiers	Ajouter, modifier, consulter et clôturer un chantier ; définir son budget total, son chef de projet et son statut (planifié, en cours, terminé).
Suivi de l'avancement	Consulter l'avancement global (%) de chaque chantier, calculé à partir des tâches.
Gestion des tâches	Créer des tâches pour un chantier, mettre à jour leur statut et leur pourcentage d'avancement.

Exigence	Description
Gestion des employés	Ajouter, modifier, consulter et désactiver les employés (nom, spécialité, téléphone, salaire journalier).
Affectation des employés	Affecter un employé à un chantier (et éventuellement à une tâche) sur une période.
Gestion des présences	Enregistrer pour chaque employé, chantier et date : présent, absent ou en retard, avec heures d'arrivée et de départ.
Gestion des matériaux	Ajouter, modifier et consulter les matériaux (nom, unité, seuil minimum).
Gestion du stock	Enregistrer les entrées et sorties par chantier ; consulter les quantités disponibles ; conserver l'historique des mouvements.
Alertes de stock	Signaler visuellement toute quantité inférieure au seuil minimum du matériau.
Gestion des machines	Enregistrer les machines, consulter et modifier leur état (opérationnelle, en maintenance, inactive, en panne) et les affecter à un chantier.
Gestion de la maintenance	Enregistrer les interventions et les dates d'entretien d'une machine.
Gestion des documents	Ajouter, consulter, modifier et supprimer les documents d'un chantier.
Suivi budgétaire	Afficher pour un chantier le budget total, le budget dépensé, le budget restant et le pourcentage utilisé.
Tableau de bord	Afficher les indicateurs du chantier : présences, inventaire des matériaux, état des machines, budget.
Rapports	Générer un rapport d'avancement, consulter les rapports existants, exporter un rapport (PDF ou CSV).

4.1. Détail des principales exigences

– Authentification

L'utilisateur saisit son adresse e-mail et son mot de passe. Si les identifiants sont valides et le compte actif, l'application ouvre la session et affiche l'interface correspondant au rôle. Sinon, un message d'erreur explicite est affiché et la session n'est pas ouverte.

– Chantiers et avancement

Un chantier possède un identifiant unique, un nom, une localisation, des dates de début et de fin prévue, un statut, un budget et un chef de projet. L'avancement du chantier est la **moyenne des pourcentages d'avancement de ses tâches** ; il est compris entre 0 et 100 %. Un chantier sans tâche a un avancement de 0 %.

– Tâches

Le pourcentage d'avancement d'une tâche doit être compris entre 0 et 100. Une tâche à 100 % passe au statut « terminée ».

– Présences

Le système enregistre la date, l'employé, le chantier, le statut (présent, absent, retard) et les heures. Une seule présence est autorisée par employé, par chantier et par jour : toute tentative de doublon est refusée avec un message explicatif. Seuls les employés affectés au chantier à cette date apparaissent dans la liste de pointage.

– Stock et alertes

Le stock est géré **par chantier et par matériau**. Après chaque entrée ou sortie validée, la quantité disponible est mise à jour et un mouvement est enregistré (matériau, quantité, type, date, utilisateur) dans la même transaction. Une sortie supérieure à la quantité disponible est refusée. Lorsque la quantité disponible est inférieure au seuil minimum, le matériau est signalé dans l'inventaire (couleur d'alerte unique et cohérente dans toute l'application).

– Machines

L'état d'une machine prend l'une des quatre valeurs : opérationnelle, en maintenance, inactive, en panne. Une machine en maintenance ou en panne ne peut pas être affectée à un chantier.

– Tableau de bord

Le tableau de bord du chantier affiche : la liste des présences du jour (présent / absent / retard), l'inventaire des matériaux (quantité disponible / quantité cible), l'état des machines et la situation budgétaire. Les valeurs proviennent exclusivement de la base de données.

5. Exigences non fonctionnelles

Exigence	Description	Critère de vérification
Sécurité	Protéger les comptes et les données.	Mots de passe hachés (BCrypt) ; requêtes SQL paramétrées (PreparedStatement) ; accès contrôlés selon les rôles.
Performance	Garantir une consultation fluide.	Opérations courantes en moins de 2 secondes en conditions normales de test ; chargements longs exécutés en tâche de fond.
Fiabilité	Éviter les incohérences et les pertes de données.	Erreurs gérées et journalisées ; opérations critiques (mouvement de stock) dans une transaction.
Intégrité	Maintenir la cohérence des données.	Clés primaires, clés étrangères, contraintes CHECK et UNIQUE appliquées en base.
Ergonomie	Faciliter l'utilisation.	Navigation latérale unique et cohérente ; formulaires lisibles ; messages compréhensibles ; interface dans une seule langue ; contrastes suffisants.
Maintenabilité	Faciliter corrections et évolutions.	Séparation en couches Présentation / Services / DAO ; tests unitaires JUnit sur les services ; journalisation Logback.
Sauvegarde	Permettre la récupération des données.	Procédure de sauvegarde et de restauration PostgreSQL documentée et testée.
Compatibilité	Fonctionner dans l'environnement prévu.	Tests sur un poste disposant de la version Java requise (voir section 6).
Traçabilité	Conserver l'historique des mouvements importants.	Chaque mouvement de stock enregistre date, type, quantité et utilisateur.
Coût abordable	Limiter le coût de réalisation.	Technologies gratuites et open source (Java, JavaFX, PostgreSQL).

Les valeurs numériques proposées, notamment le délai de réponse, sont des objectifs à confirmer pendant les tests.

6. Contraintes techniques

Domaine	Choix retenu	Justification
Langage	Java 17 ou supérieur	Version à support long ; langage étudié et orienté objet.
Interface	JavaFX (FXML + CSS)	Séparation vue / logique ; maquettes réalisables fidèlement avec CSS.
Architecture	3 couches, patron MVC	Maintenabilité (ENF06).
Base de données	PostgreSQL 14 ou supérieur (base chantier_db)	Intégrité relationnelle, contraintes CHECK, transactions, gratuit.
Accès aux données	JDBC, PreparedStatement, pool de connexions HikariCP	Contrôle du SQL, protection contre l'injection.
Sécurité	BCrypt pour le hachage des mots de passe	Standard reconnu.
Journalisation / tests	Logback ; JUnit 5	Traçabilité des erreurs ; tests des règles métier sans interface.
Outils de conception	Figma (maquettes), draw.io / Mermaid (UML)	Prototypage et diagrammes partageables.
Gestion de projet	Git et GitHub	Versionnement du code et des documents.
Déploiement	Application desktop installée sur le poste ; serveur PostgreSQL local ou en réseau local	Pas de service web à héberger.

7. Règles de gestion

Règle
Chaque chantier possède un identifiant unique.
Chaque utilisateur, employé, matériau et machine possède un identifiant unique ; l'e-mail d'un utilisateur est unique.
La progression d'une tâche est comprise entre 0 et 100 %.
L'avancement d'un chantier est la moyenne des avancements de ses tâches.
La quantité disponible d'un matériau sur un chantier ne doit jamais être négative.
Chaque mouvement de stock précise le matériau, le chantier, la quantité (> 0), le type (entrée ou sortie), la date et l'utilisateur.
Une machine en maintenance ou en panne ne peut pas être affectée à un chantier.
Un utilisateur n'accède qu'aux fonctionnalités autorisées par son rôle.
Une présence est associée à un employé, un chantier et une date ; il ne peut exister qu'une présence par employé, chantier et jour.
Les suppressions de données liées à un historique (mouvements de stock, présences) sont interdites ; on désactive ou on clôture à la place.
Les opérations modifiant plusieurs données (ex. stock + mouvement) s'exécutent dans une transaction.
Un chantier est dirigé par au plus un chef de projet à la fois et supervisé par un chef de département ; le budget dépensé ne peut pas être négatif.

8. Données principales du système

Les entités ci-dessous sont celles du diagramme de classes v2 et des 14 tables de la base PostgreSQL.

Entité	Rôle dans le système	Table
Utilisateur (ChefDepartement, ChefProjet)	Compte de connexion avec rôle.	utilisateur
Chantier	Site de construction, budget, statut, avancement.	chantier
Tache	Travail à réaliser sur un chantier.	tache
Document	Fichier lié à un chantier.	document
Rapport	Rapport d'avancement généré pour un chantier.	rapport
Employe	Personnel de chantier (ouvrier).	employe
AffectationEmploye	Période d'affectation d'un employé à un chantier (et une tâche).	affectation_employe
Presence	Pointage quotidien d'un employé sur un chantier.	presence
Materiau	Catalogue des matériaux (unité, seuil minimum).	materiau
StockChantier	Quantité d'un matériau disponible sur un chantier.	stock_chantier
MouvementStock	Entrée ou sortie de stock (historique).	mouvement_stock
Machine	Équipement et son état.	machine
AffectationMachine	Période d'affectation d'une machine à un chantier.	affectation_machine
Maintenance	Intervention d'entretien sur une machine.	maintenance

Vocabulaire unifié : « Ouvrier » et « Worker » deviennent **Employé** ; « Materiel » et « Material » deviennent **Matériau** ; « Site manager » devient **Chef de projet**.

9. Vérification des exigences

Scénario de test	Résultat attendu
Connexion avec des identifiants valides.	Accès autorisé, interface selon le rôle.
Connexion avec un mot de passe erroné.	Accès refusé, message d'erreur affiché.
Création d'un chantier avec les informations obligatoires.	Chantier enregistré et visible dans la liste.
Saisie d'une progression de tâche supérieure à 100 %.	Saisie refusée avec message explicatif.
Chantier avec deux tâches à 100 % et 50 %.	Avancement du chantier = 75 %.
Sortie de stock supérieure à la quantité disponible.	Opération refusée, stock inchangé.
Entrée de stock valide.	Quantité mise à jour et mouvement historisé.
Quantité disponible inférieure au seuil minimum.	Matériau signalé en alerte.
Affectation d'une machine en maintenance.	Affectation refusée.
Enregistrement d'une présence valide.	Présence enregistrée et consultable.
Deuxième présence pour le même employé, chantier et jour.	Refusée (doublon).
Tentative d'accès d'un chef de projet à la gestion des utilisateurs.	Accès refusé.
Consultation du tableau de bord.	Indicateurs affichés à partir des données enregistrées.
Génération puis export d'un rapport.	Rapport enregistré et fichier exporté.

10. Matrice de traçabilité

Chaque exigence fonctionnelle est reliée à un cas d'utilisation, à un écran (maquette) et aux classes du modèle. Les écrans marqués « à créer » sont listés dans le dossier de conception.

Écran	Classes / tables
Login (existe)	Utilisateur
Users (à créer)	Utilisateur
Projects (existe), formulaire chantier (à créer)	Chantier
Project detail (existe)	Chantier, Tache
Tasks (à créer)	Tache
Employees (à créer)	Employe
Assignments (à créer)	AffectationEmploye
Attendance (existe, à compléter)	Presence, Employe
Materials (à créer)	Materiau
Stock (à créer)	StockChantier, MouvementStock
Dashboard, Stock	StockChantier, Materiau
Machines (à créer)	Machine, AffectationMachine
Machines (à créer)	Maintenance
Documents (à créer)	Document
Dashboard, Project detail (existent)	Chantier
Dashboard (existe)	Toutes les classes de suivi
Reports (à créer)	Rapport

11. Conclusion

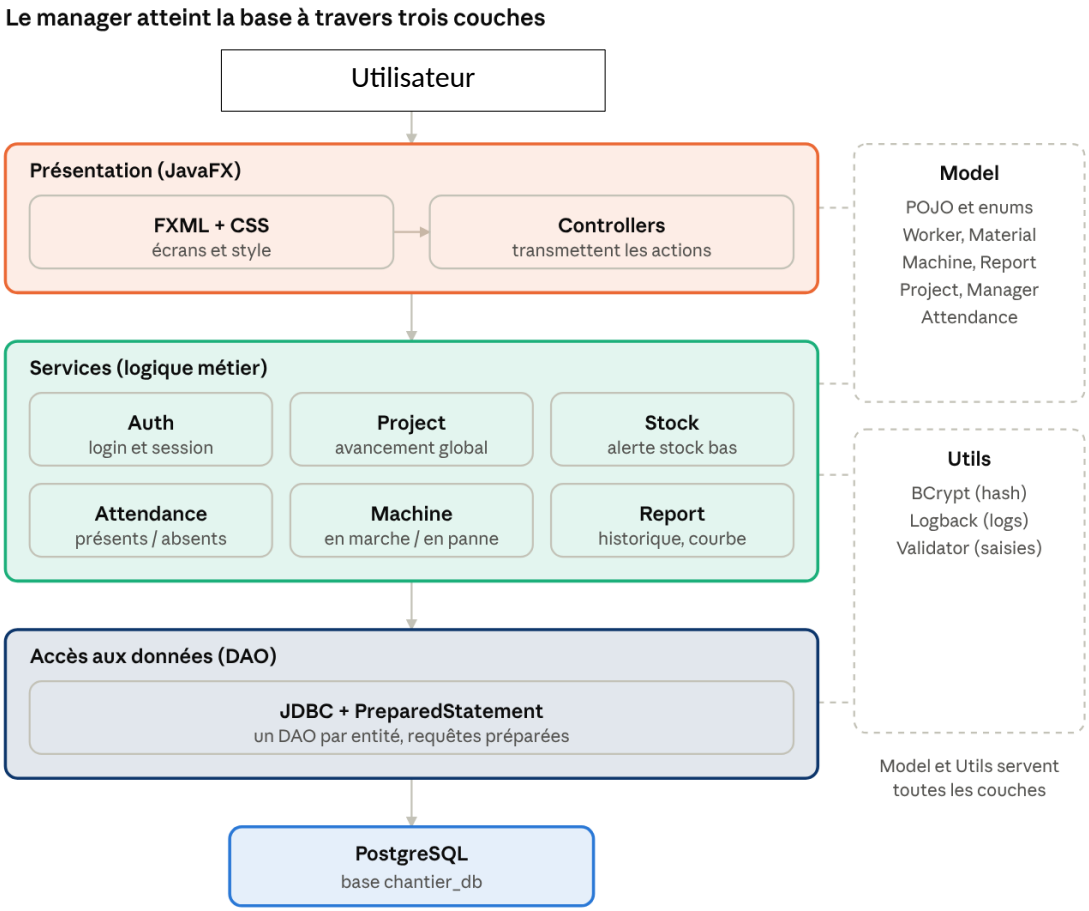
Ce document constitue la référence des exigences pour le développement de l'application. Il guide la conception UML, la réalisation des maquettes, la création de la base PostgreSQL et les tests de validation. Toute modification d'une exigence doit être répercutée dans les diagrammes, l'architecture et les maquettes via les identifiants EF, ENF et RG.

CONSTRUCTION SITE MANAGMENT : Architecture de l'application

Application desktop Java (JavaFX) de gestion de chantiers, avec PostgreSQL. Utilisateur unique : le site manager (chef de chantier).

1. Vue d'ensemble

L'application suit une architecture en **3 couches** basée sur le patron **MVC**. Chaque couche a une seule responsabilité. Le sens des appels est unique :



Règle principale : Controller → Service → DAO → Base de données. Le Controller ne parle jamais directement au DAO ni à la base.

2. Site manager

C'est l'**utilisateur unique** de l'application. Il se connecte, consulte le stock, pointe la présence, suit les machines et crée les rapports d'avancement.

3. Couche Présentation (JavaFX)

Elle affiche les écrans au manager.

Élément	Rôle	Exemple
FXML + CSS	FXML décrit la structure de l'écran (boutons, tableaux, champs). CSS définit les couleurs et le style.	`stock.fxml` contient le tableau des matériaux et le bouton « Ajouter »
Controllers	Classes Java qui reçoivent les actions du manager (clic, saisie) et appellent les Services.	`StockController` envoie les données à `StockService`

Règle : le Controller ne contient ni logique métier ni SQL. Il transmet seulement les données entre l'écran et le service.

4. Couche Services (logique métier)

C'est le « cerveau » de l'application : règles et vérifications. Chaque service gère un module.

Service	Responsabilités
Auth	Vérifie identifiant et mot de passe (BCrypt), gère la session
Project	Fournit l'avancement global du chantier (%)
Stock	Ajoute, modifie, supprime les matériaux ; détecte les stocks sous le seuil (alerte)
Attendance	Enregistre présent/absent ; calcule « 12/15 présents »
Machine	Calcule total / en marche / en panne ; change l'état
Report	Crée un rapport ; fournit l'historique pour la courbe de progression

Pourquoi une couche Service ? Les règles (ex. « la quantité ne peut pas être négative ») sont écrites à un seul endroit et testées avec JUnit sans interface. Cela répond à l'exigence **Maintenabilité (NF5)**.

5. Couche Accès aux données (DAO)

DAO (Data Access Object) est la **seule couche qui communique avec la base**.

- Un DAO par entité : `MaterialDAO`, `WorkerDAO`, `MachineDAO`, etc.
- Accès via **JDBC** et **PreparedStatement** : protection contre l'injection SQL (**Sécurité NF3**).
- Méthodes : `save()`, `update()`, `delete()`, `findAll()`, `findById()`.

6. Base de données PostgreSQL

Stockage permanent des données dans la base `chantier_db` : tables `manager`, `project`, `worker`, `attendance`, `material`, `machine`, `report`. Les clés étrangères et contraintes `CHECK` garantissent l'intégrité (ex. avancement entre 0 et 100).

7. Éléments transversaux

Élément	Contenu	Utilité
Model	Classes POJO (`Worker`, `Material`, `Machine`, `Report`, `Project`) et enums (`MachineStatus`, `AttendanceStatus`)	Représentent les données ; utilisées par toutes les couches
Utils : BCrypt	Hachage des mots de passe	Sécurité (NF3)
Utils : Logback	Journalisation des erreurs et actions	Maintenabilité (NF5)
Utils : Validator	Validation des saisies (champ vide, nombre négatif)	Fiabilité des données

Ces éléments ne sont pas une couche : toutes les couches peuvent les utiliser.

8. Exemple complet : ajouter du ciment au stock

1. Le manager remplit l'écran Stock et clique sur **Ajouter** (FXML).
2. `StockController` récupère les données et appelle `StockService.addMaterial()`.
3. `StockService` vérifie les données (Validator) : nom non vide, quantité ≥ 0.
4. `StockService` appelle `MaterialDAO.save()`.
5. `MaterialDAO` exécute un `INSERT` avec PreparedStatement dans PostgreSQL.
6. Le résultat remonte au Controller ; le tableau est mis à jour et, si la quantité est inférieure au seuil, une alerte rouge s'affiche.

9. Lien avec les exigences non fonctionnelles

Exigence	Réponse de l'architecture
Utilisabilité	Écrans simples, navigation latérale unique
Performance	Index en base, chargement en tâche de fond (`Task` JavaFX)
Sécurité	BCrypt, PreparedStatement, validation, session
Coût abordable	Java, JavaFX et PostgreSQL sont gratuits et open-source
Maintenabilité	Couches séparées, tests JUnit mensuels, logs Logback

5. Diagrammes UML

5.1 Diagramme de cas d'utilisation

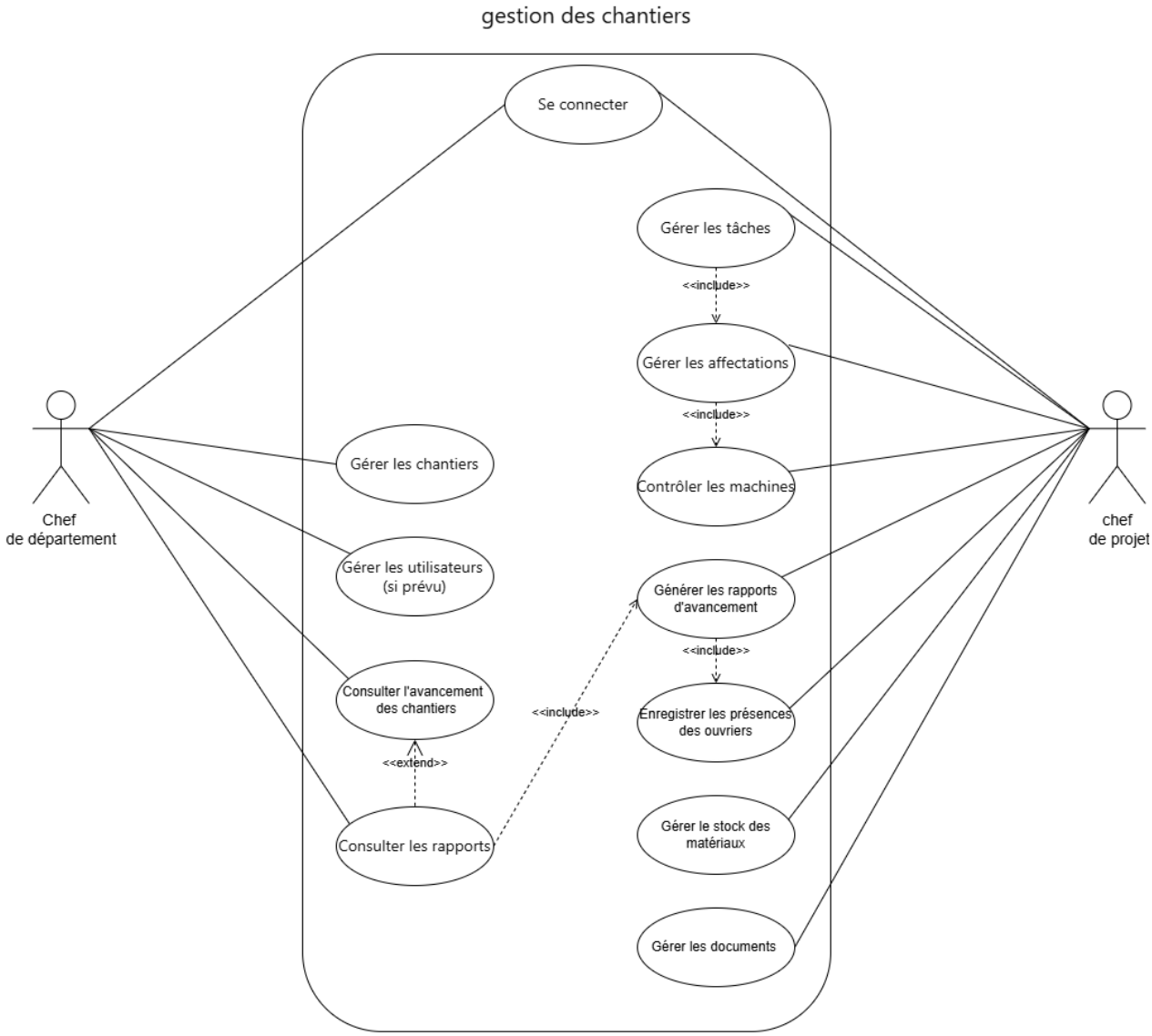


Figure 1 : Diagramme de cas d'utilisation, gestion des chantiers

5.2 Diagramme de classes

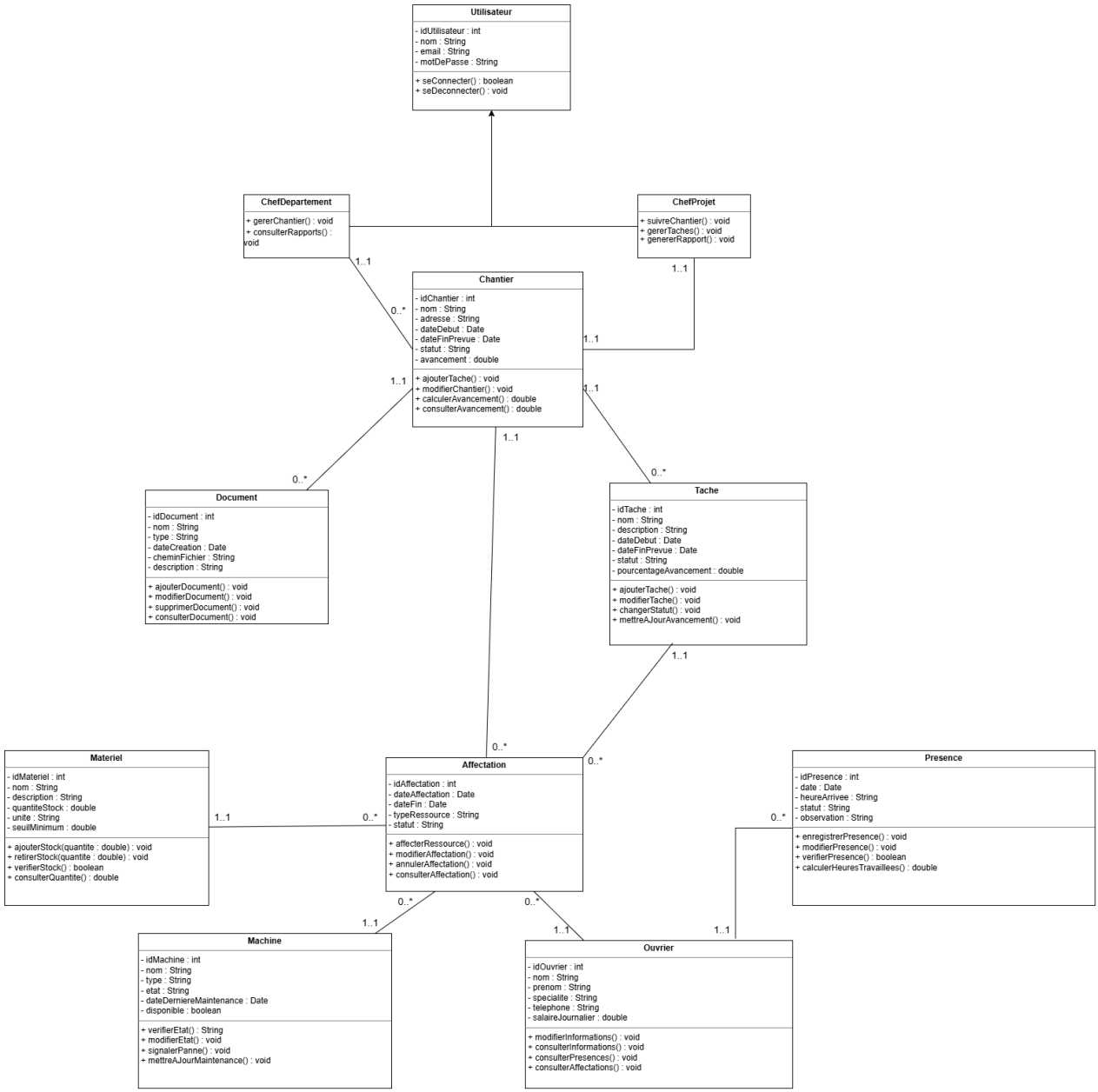


Figure 2 : Diagramme de classes

5.3 Diagrammes de séquence

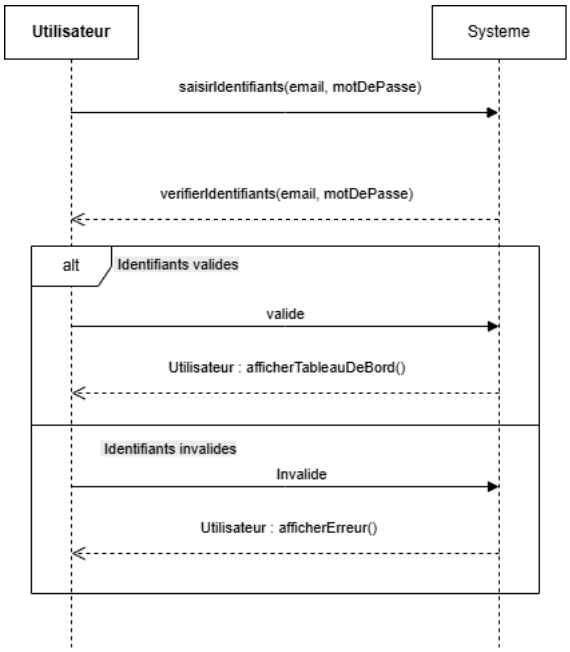


Figure 3 : Séquence, authentification

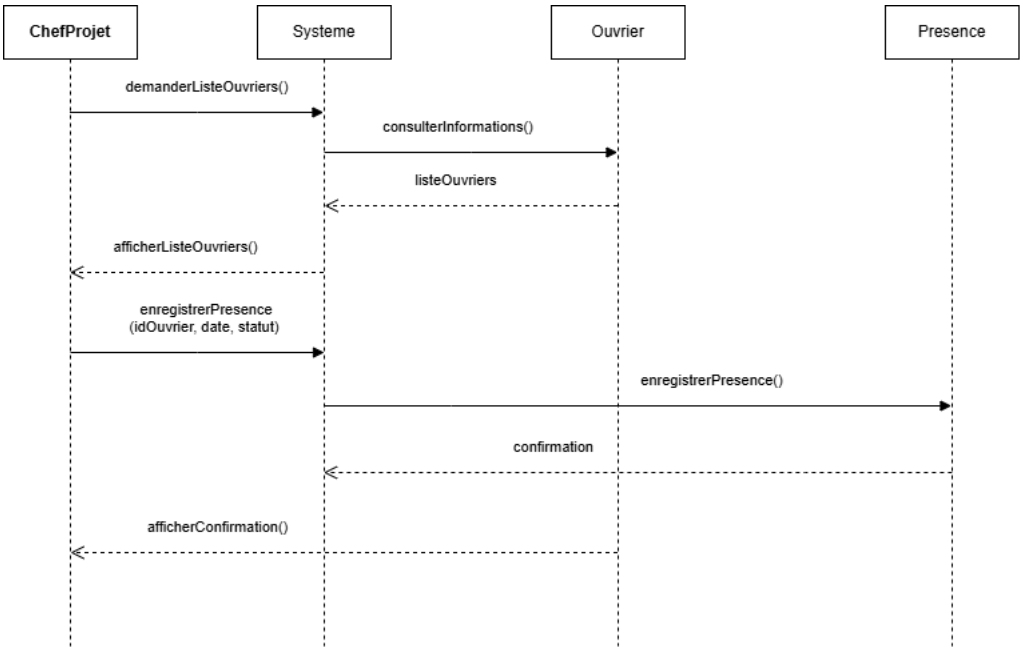


Figure 4 : Séquence, enregistrement des présences

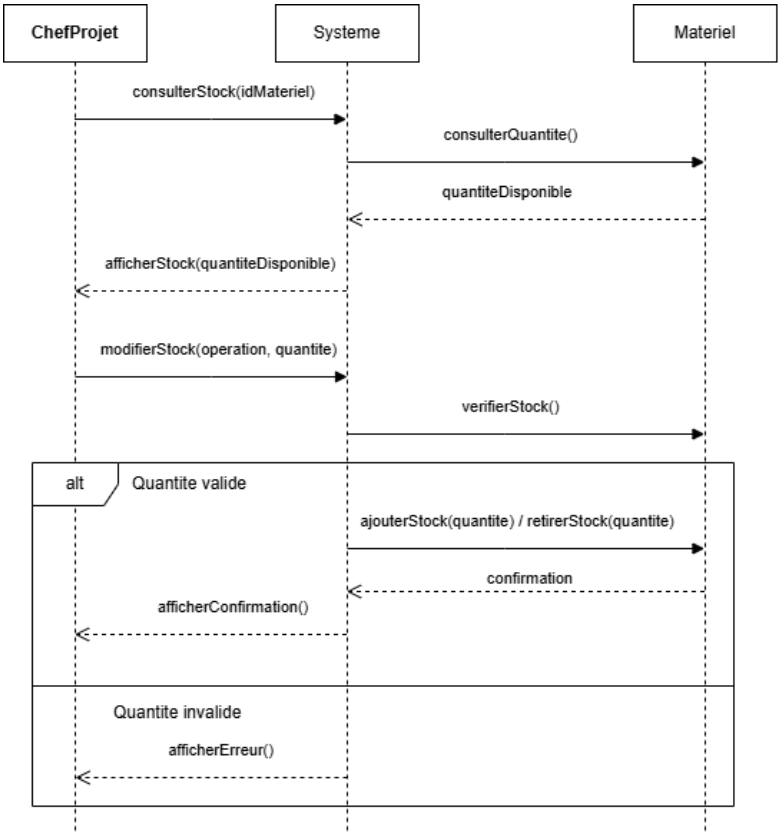


Figure 5 : Séquence, consultation et modification du stock

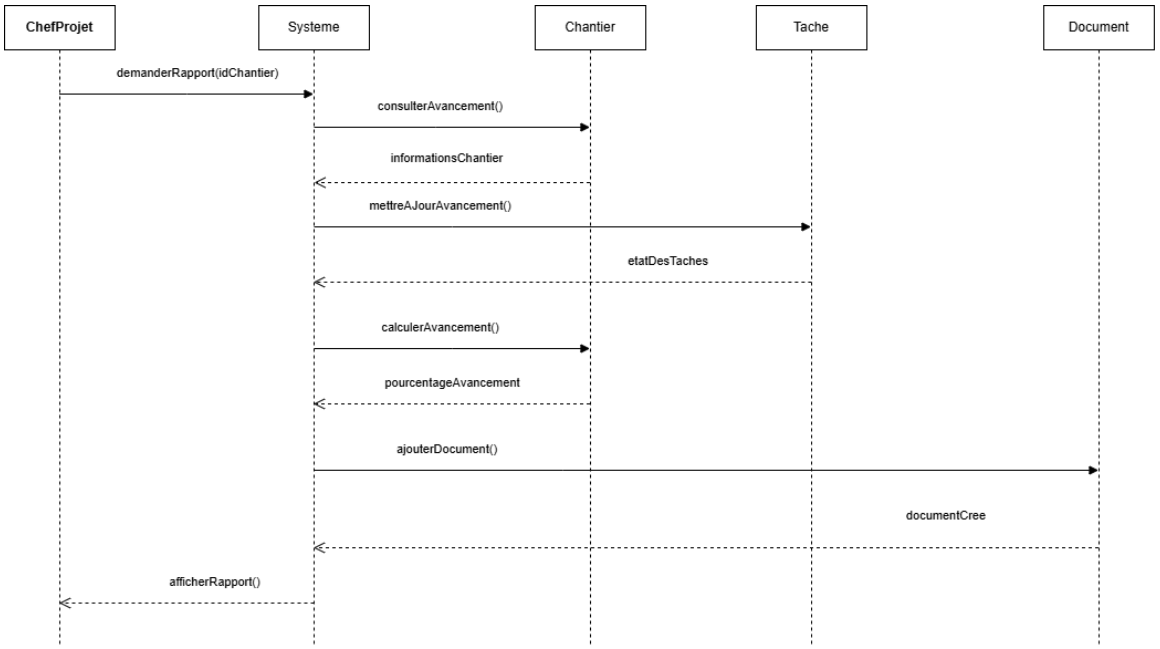


Figure 6 : Séquence, génération d'un rapport d'avancement

6. Maquettes de l'interface

Maquettes réalisées sous Figma : écran de connexion, tableau de bord, liste des projets, détail d'un projet et pointage des présences. Les thèmes sombre et clair donnent à l'application une identité cohérente.

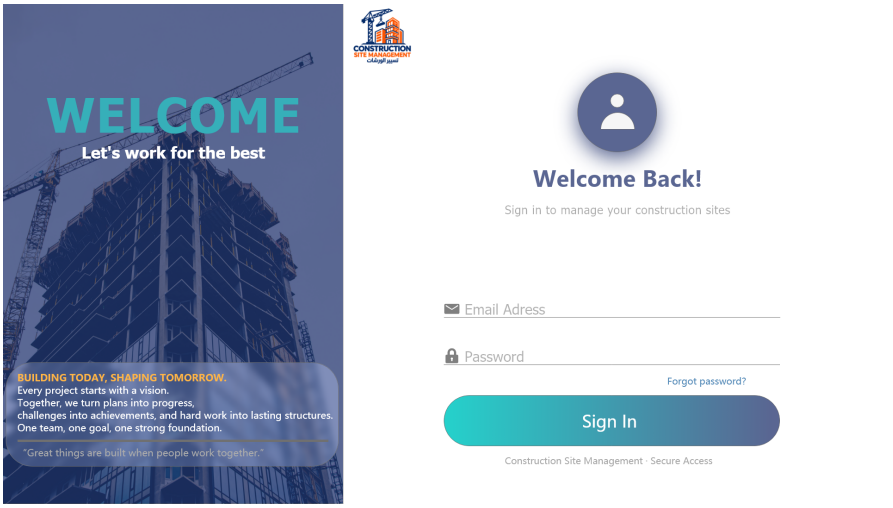


Figure 7 : Écran de connexion

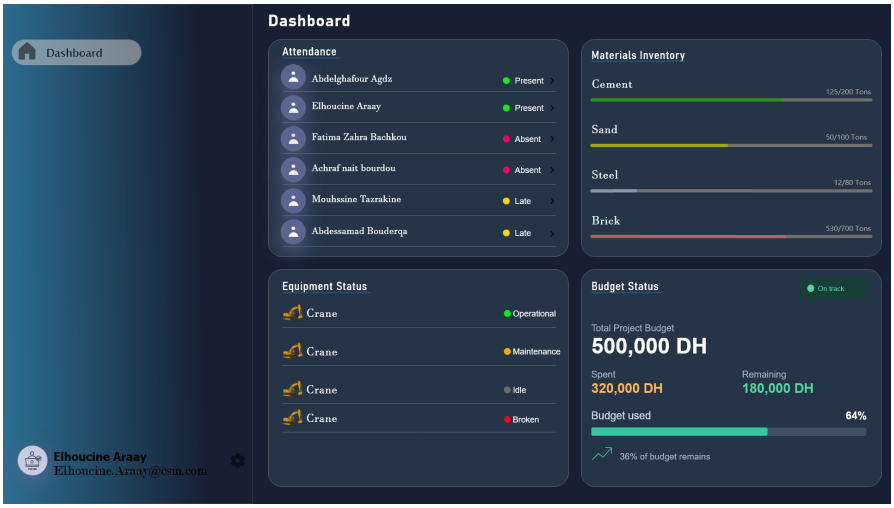


Figure 8 : Tableau de bord

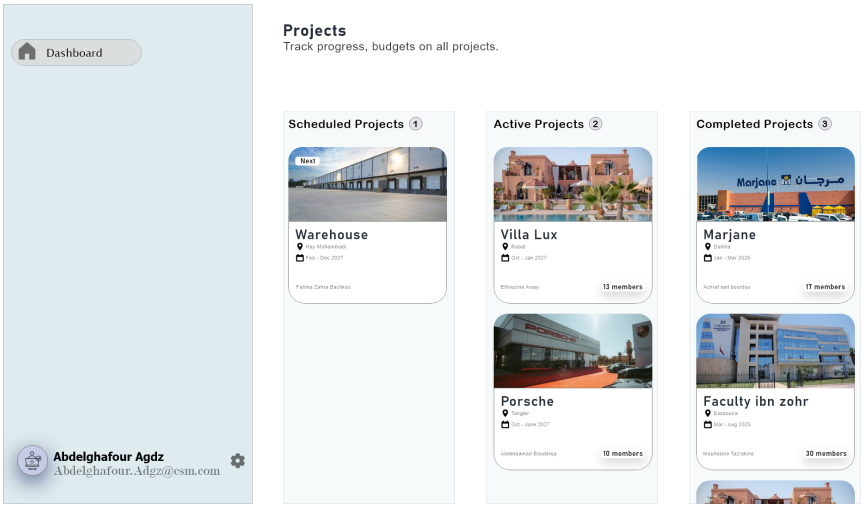


Figure 9 : Liste des projets (planifiés, actifs, terminés)



Figure 10 : Détail d'un projet

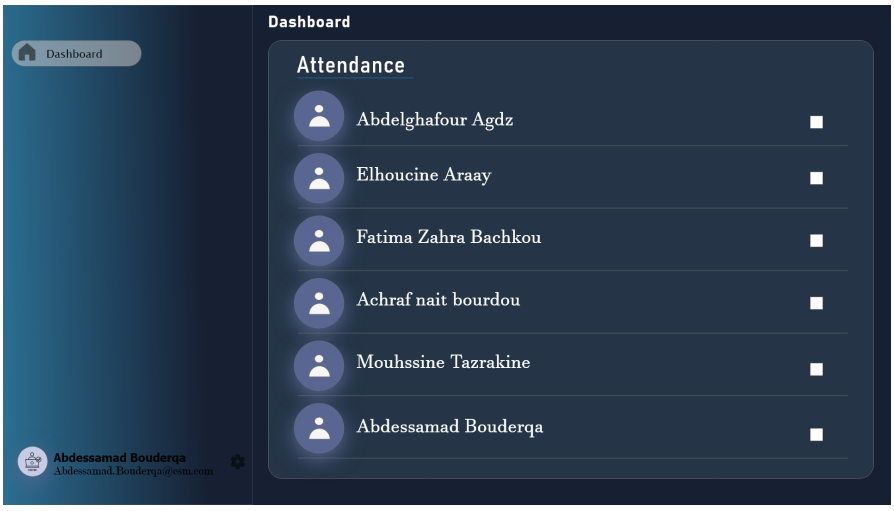


Figure 11 : Pointage des présences